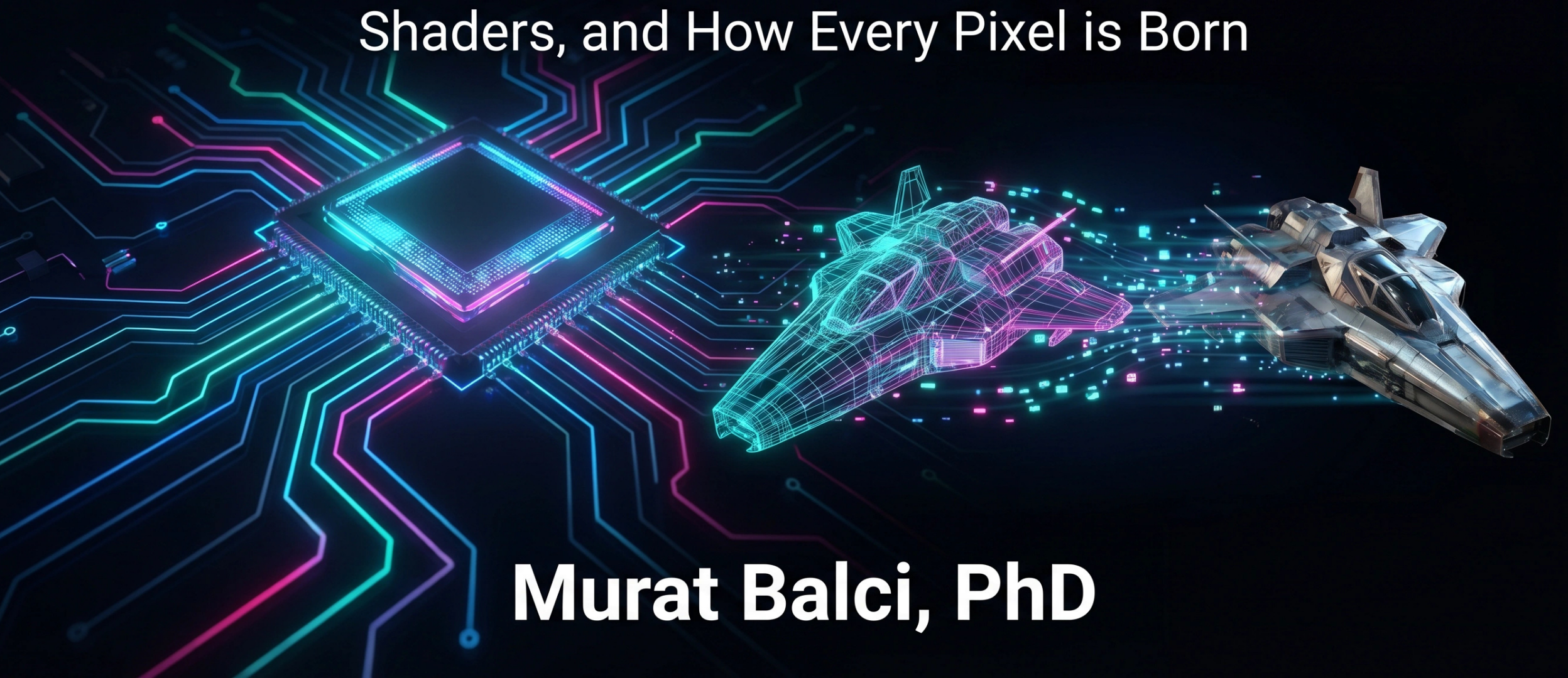


How GPUs Work: From Instructions to Pixels

GPU Architecture, the Rendering Pipeline, Compute Shaders, and How Every Pixel is Born



Murat Balci, PhD

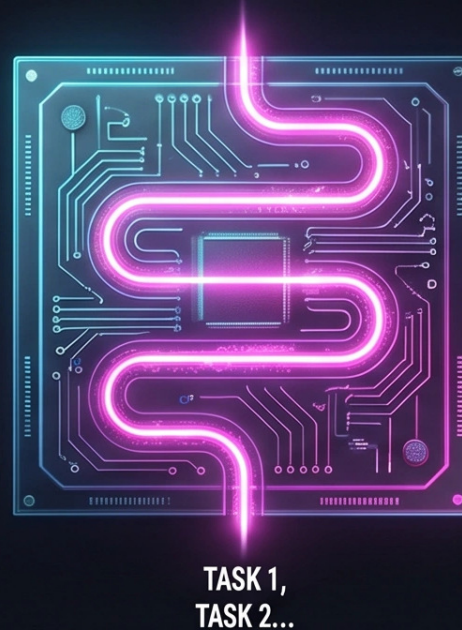
WHY GPUs EXIST: THE PARALLEL PROCESSING IMPERATIVE.

The necessity of parallel architecture for rendering and complex computations.

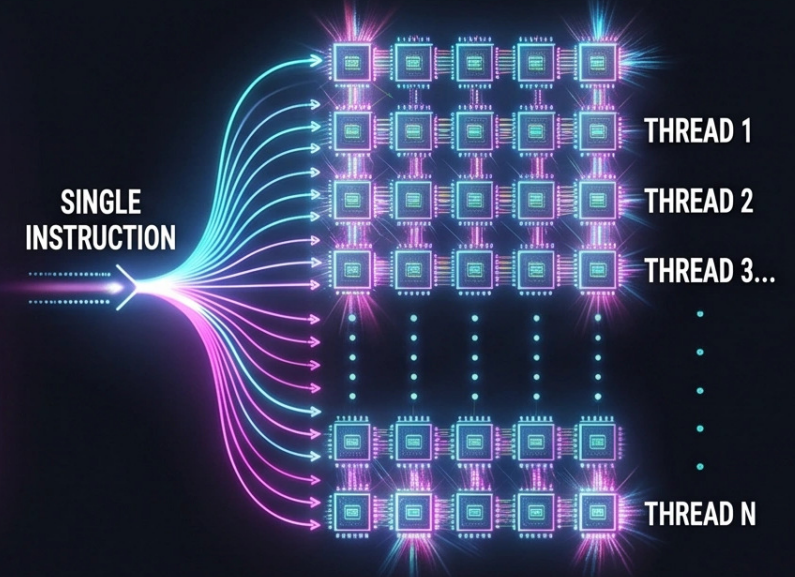
- 1. Rendering a 1080p frame requires computing color for 2 million pixels independently.
- 2. CPUs excel at complex sequential logic; GPUs excel at simple parallel operations.
- 3. Modern GPUs contain thousands of shader cores organized into Streaming Multiprocessors (SMs).
- 4. SIMT (Single Instruction, Multiple Threads) architecture makes GPUs ideal for graphics and compute.

CPU VS. GPU: PROCESSING PARADIGMS

CPU: SEQUENTIAL PROCESSING



GPU: PARALLEL PROCESSING (SIMT)



CPU vs. GPU: ARCHITECTURAL PHILOSOPHY

Understanding the Fundamental Differences in Processing Power and Design.



1. **CPU**: 4-16 LARGE, COMPLEX CORES with DEEP PIPELINES, LARGE CACHES — OPTIMIZED FOR **LATENCY** and **SERIAL THROUGHPUT**.



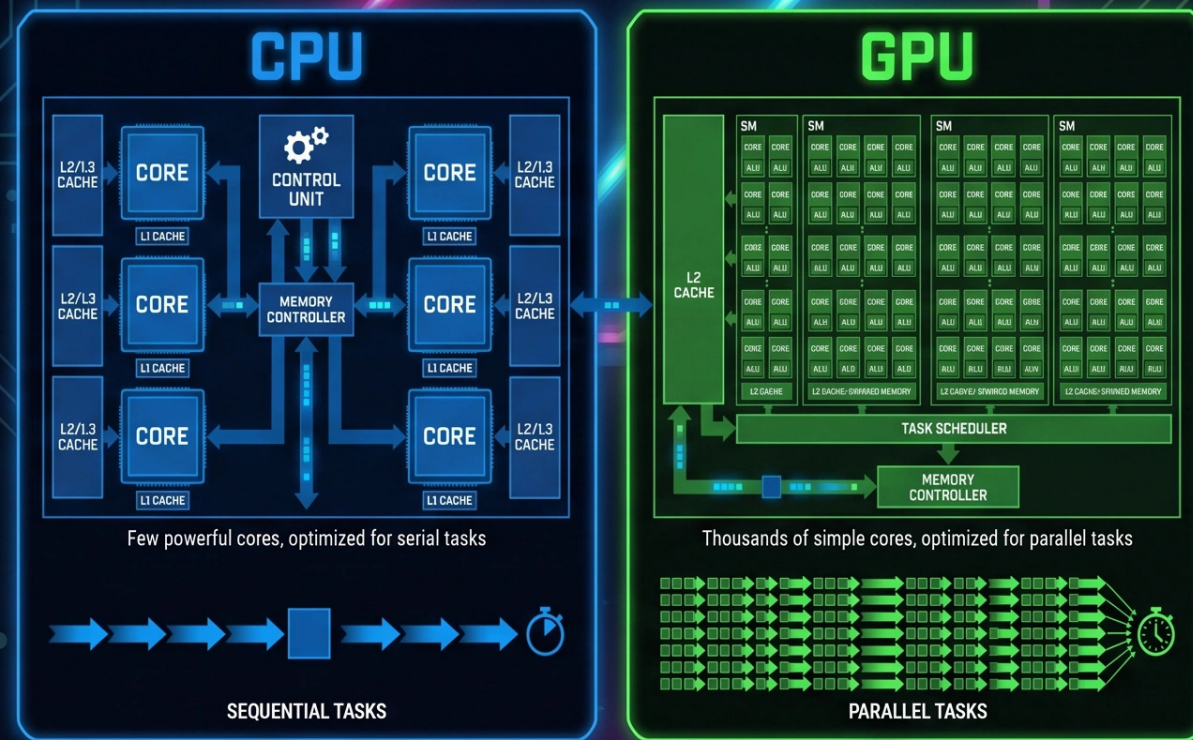
2. **GPU**: HUNDREDS TO THOUSANDS OF SMALL, SIMPLE CORES with MINIMAL CACHE — OPTIMIZED FOR **THROUGHPUT** and **PARALLELISM**.



3. **CPU** DEVOTES ~50% OF TRANSISTORS TO CONTROL LOGIC/CACHE; **GPU** DEVOTES ~80% TO ALUs.



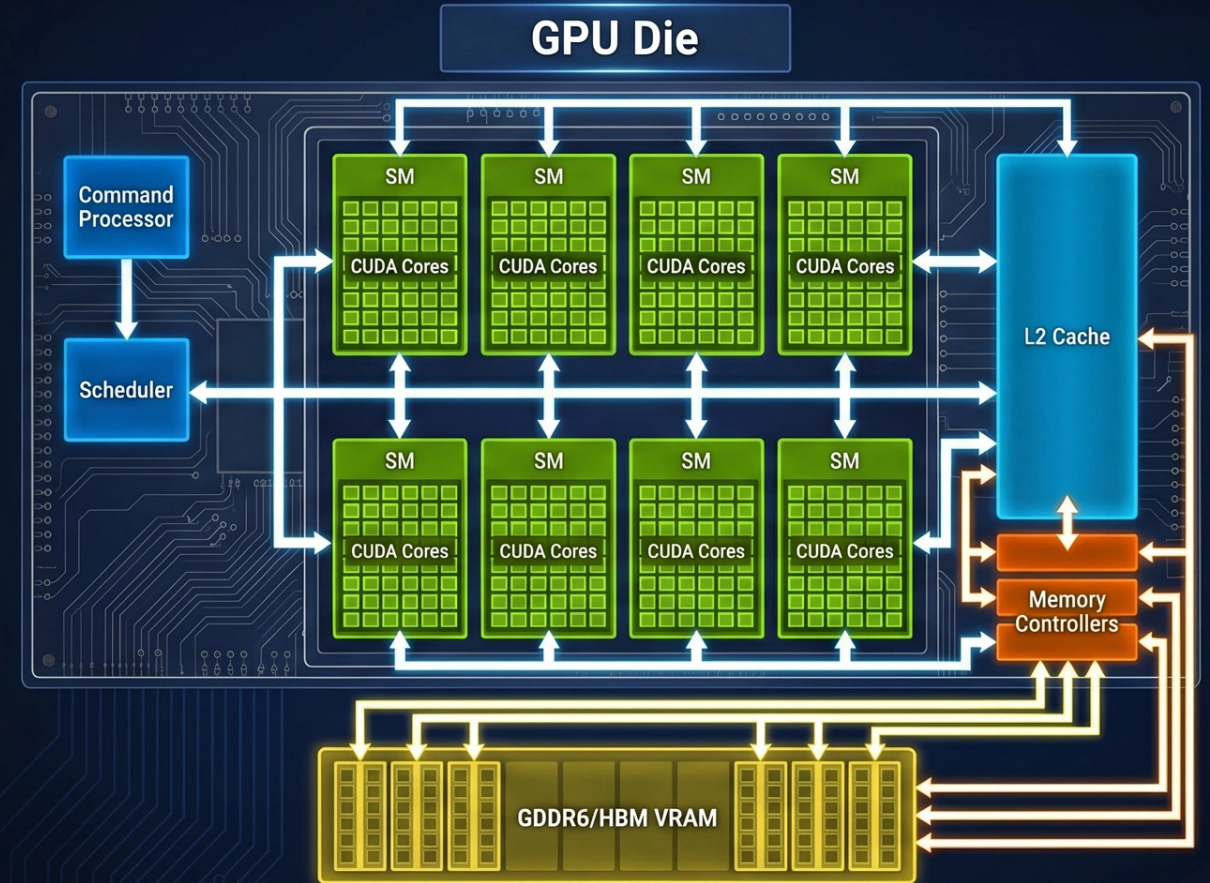
4. **GPUs** HIDE MEMORY LATENCY THROUGH **MASSIVE THREAD PARALLELISM**.



GPU Hardware Architecture: Inside the Chip

KEY POINTS:

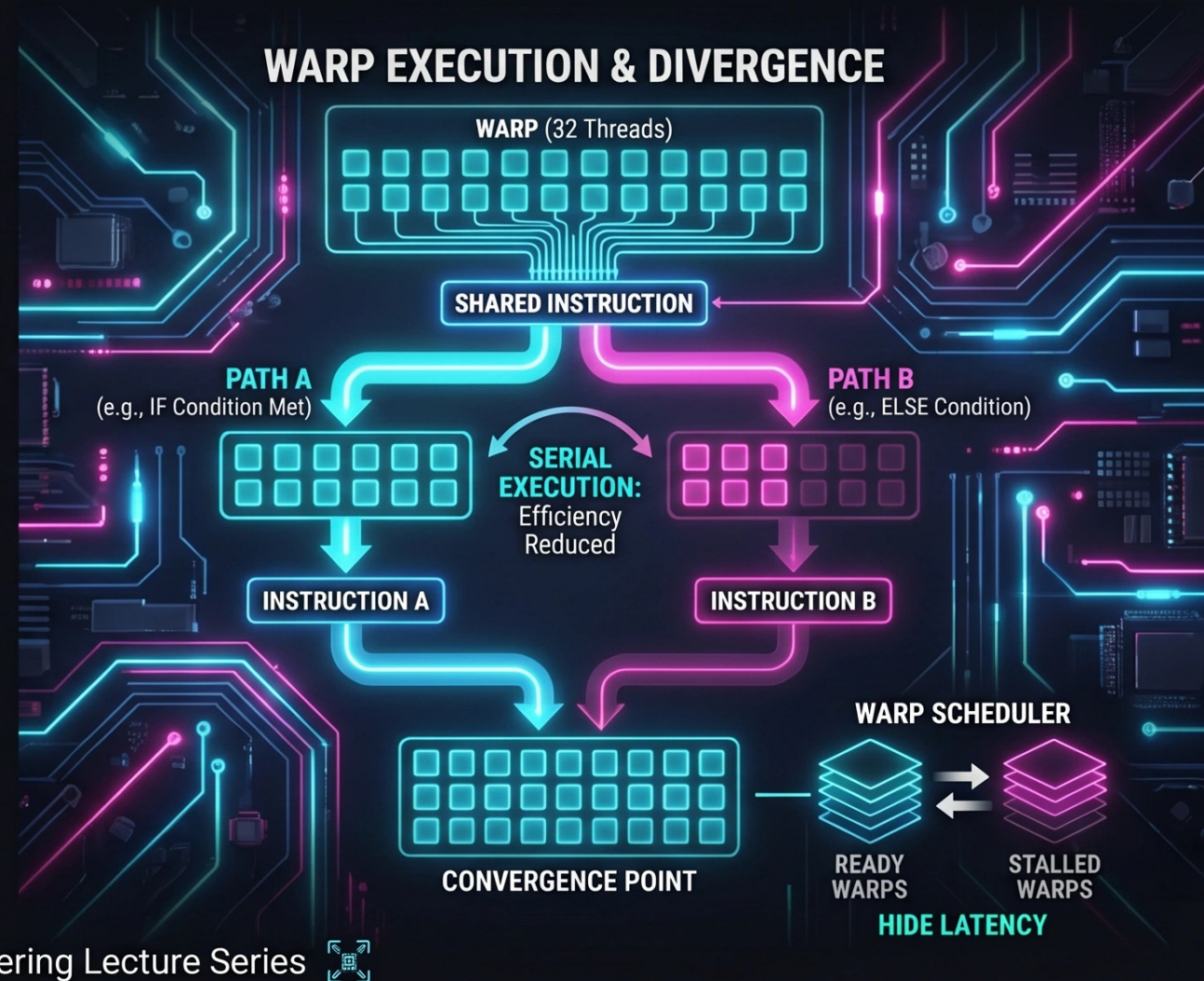
- **GPU Processing Clusters (GPCs):** Top-level groupings containing multiple SMs and a rasterizer.
- **Streaming Multiprocessors (SMs):** The fundamental compute unit containing CUDA cores, tensor cores, and warp schedulers.
- **CUDA Cores:** Individual ALUs executing integer and floating-point operations.
- **Memory Hierarchy:** Registers -> L1/Shared Memory -> L2 Cache -> GDDR6/HBM VRAM.
- **Command Processor:** Receives draw calls and distributes work.



WARP EXECUTION: HOW THREADS ACTUALLY RUN

Understanding Parallel Instruction Execution, Branch Divergence, and Latency Hiding.

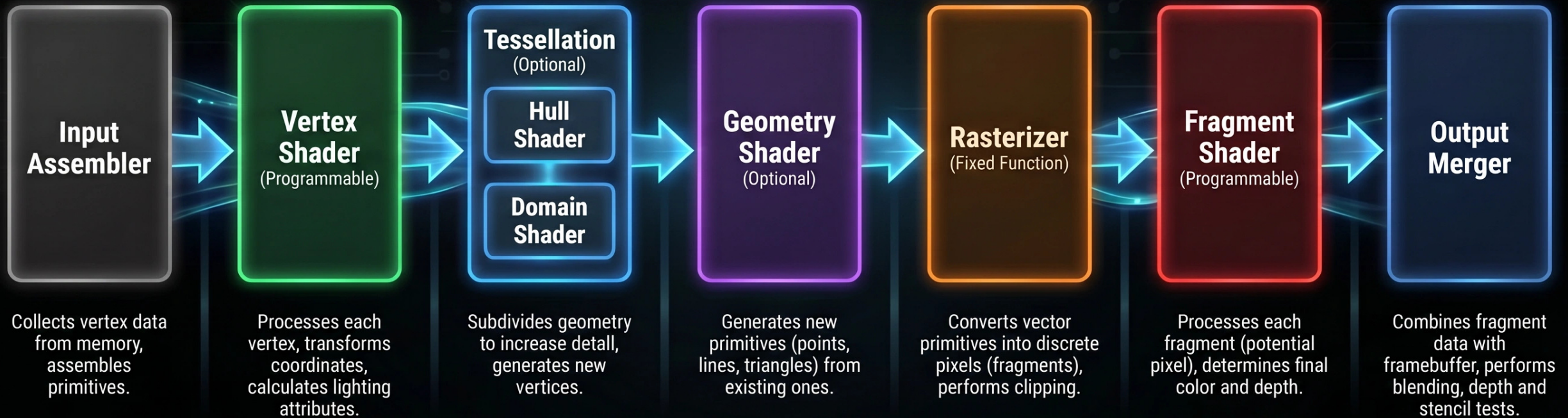
- A **warp** (NVIDIA) or **wavefront** (AMD) is a group of 32 (or 64) **threads** executing the same instruction simultaneously. 
- All threads in a warp share the same program counter, executing in lockstep. 
- **Branch divergence** occurs when threads take different paths, executing serially and reducing efficiency. 
- The warp scheduler **hides latency** by switching between ready warps at zero cost. 
- Thousands of warps can be in-flight simultaneously. 



THE GRAPHICS PIPELINE: 7 STAGES FROM DATA TO DISPLAY

KEY POINTS:

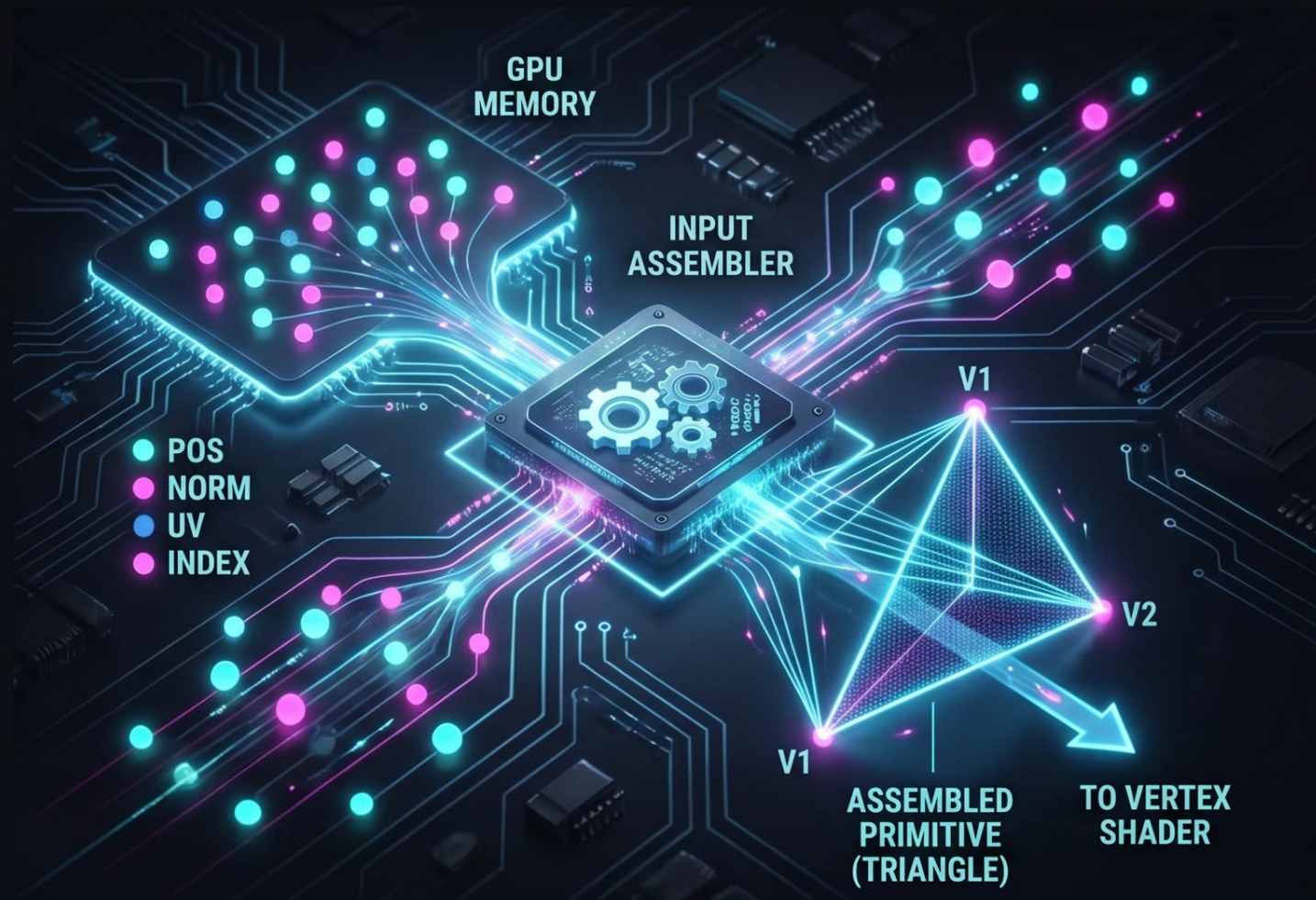
- 1. Fixed sequence of stages transforming 3D geometry into a 2D image.
- 2. Stages alternate between programmable (shader code) and fixed-function (hardware).
- 3. Pipeline: Input Assembler -> Vertex Shader -> Tessellation -> Geometry Shader -> Rasterizer -> Fragment Shader -> Output Merger.
- 4. Processes millions of vertices and billions of fragments per second.
- 5. APIs: Vulkan, DirectX 12, Metal, OpenGL.



Stage 1: Input Assembler – Feeding the Pipeline

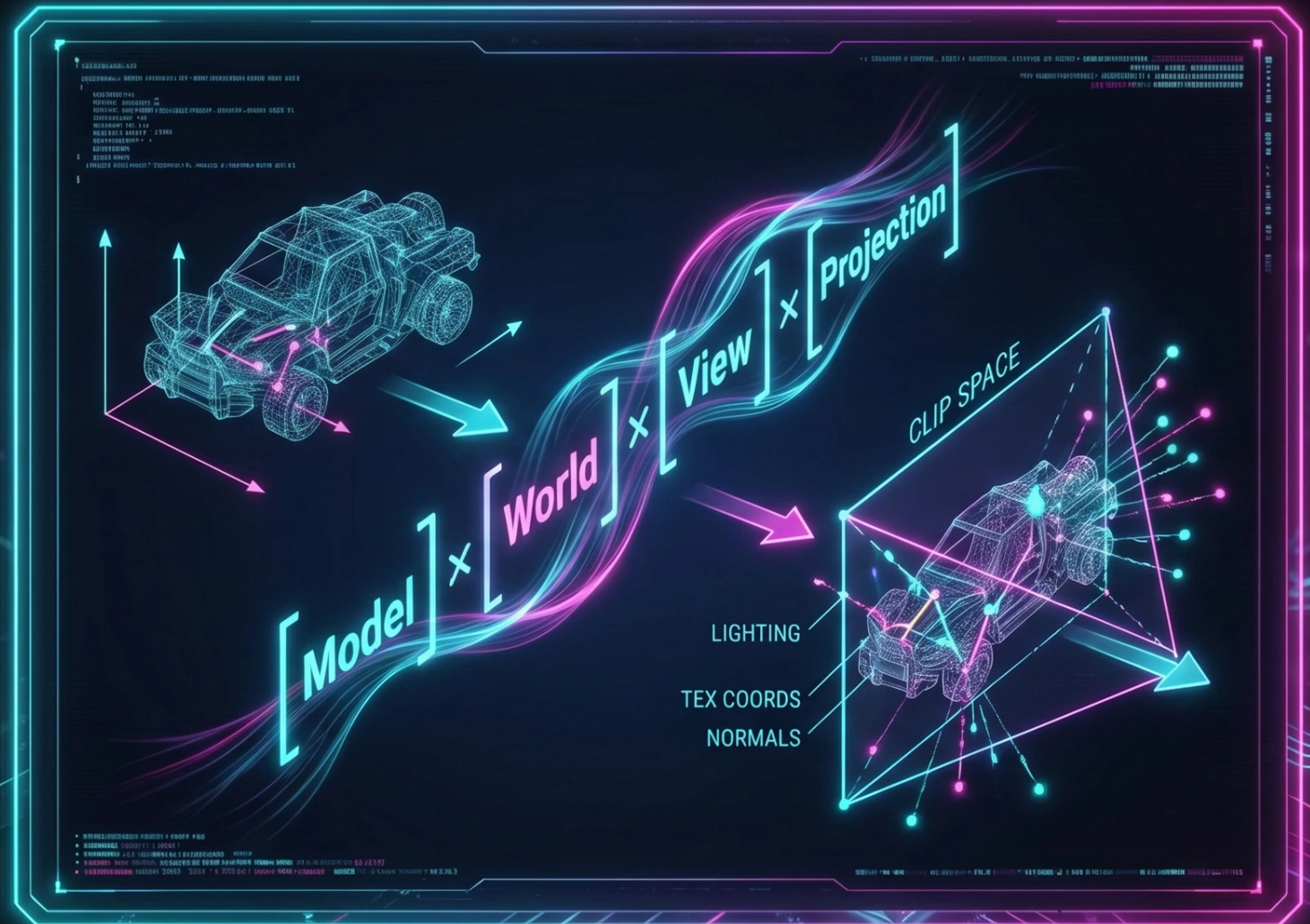
The Foundation of the Rendering Pipeline.

- **Reads Buffers from Memory:**
Fetches vertex buffers (position, normal, UV) and index buffers directly from GPU memory.
- **Assembles Primitives:**
Combines raw vertex data into fundamental geometric shapes: triangles, lines, or points.
- **Defines Connectivity:**
Uses triangle lists, strips, and fans; index buffers significantly reduce vertex duplication.
- **Fixed-Function Stage:**
A dedicated hardware process with no programmable shader code execution.
- **Feeds Vertex Shader:**
The assembled primitives are passed down the pipeline for transformation and processing.



STAGE 2: VERTEX SHADER – TRANSFORMING EVERY VERTEX.

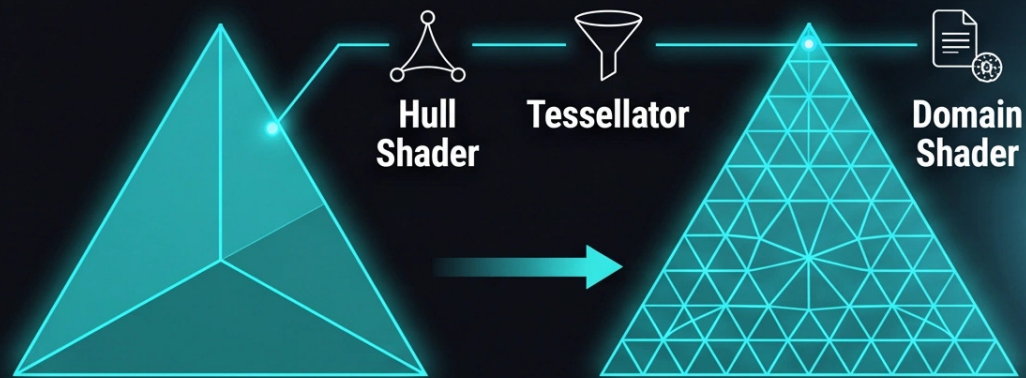
- 1. A **programmable stage**: developer-written code executes once per vertex.
- 2. **Primary job**: transform vertex positions through the Model $\rightarrow$ World $\rightarrow$ View $\rightarrow$ Projection matrix chain.
- 3. **Computes per-vertex attributes**: lighting, texture coordinates, normals, tangent vectors.
- 4. **Output**: vertices in clip space ready for clipping and projection.
- 5. Enables effects like skeletal animation, vertex displacement, and procedural geometry.



STAGE 3-4: TESSELLATION AND GEOMETRY SHADERS

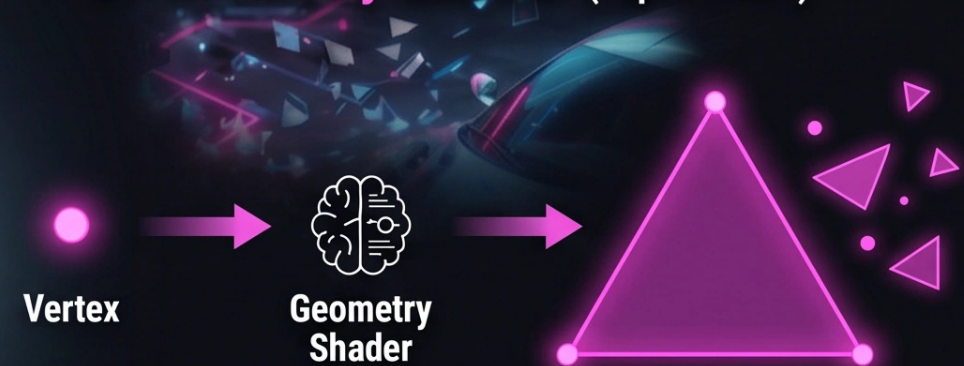
GPU Pipeline Enhancements for Detail and Flexibility.

1. Tessellation (Optional)



- **Subdivides** coarse geometry into finer detail.
- Used for terrain LOD (Level of Detail) and displacement mapping.

2. Geometry Shader (Optional)



- Processes entire primitives (points, lines, triangles).
- Can emit new primitives or discard existing ones. Used for particle systems and shadow volumes.

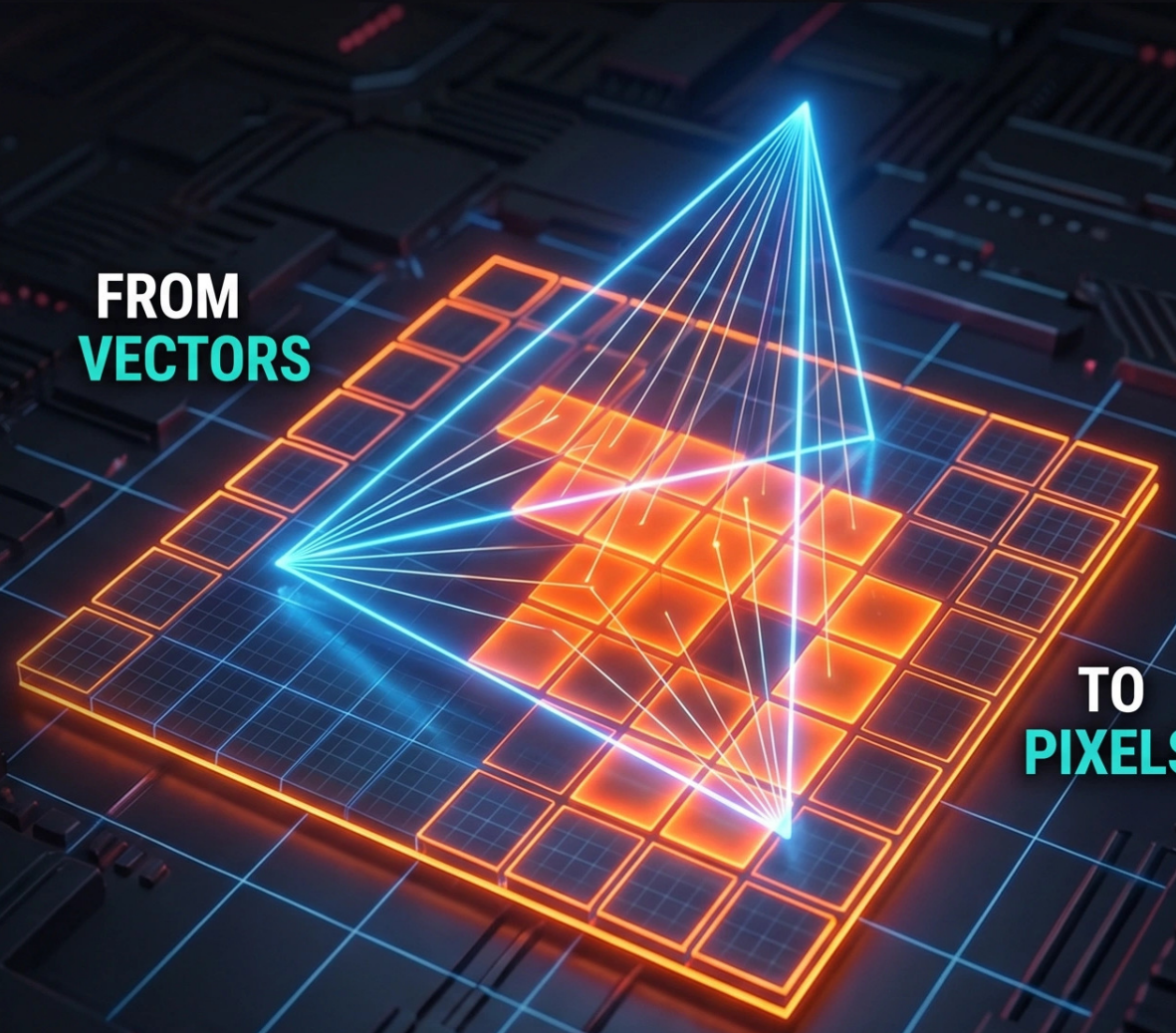
3. Both stages are optional and often skipped in performance-critical paths.

STAGE 5: **RASTERIZER** — FROM VECTORS TO PIXELS.

- A **fixed-function** stage that converts continuous geometric primitives into **discrete fragments** (potential pixels).
- **Determines** which screen pixels each triangle covers using edge equations and scanline traversal.
- **Performs perspective-correct interpolation** of vertex attributes (color, UV, depth) across the triangle face.
- **Handles clipping** (removing geometry outside the view frustum) and **back-face culling** (discarding triangles facing away).
- Each generated fragment carries interpolated attributes and enters the fragment shader.

**FROM
VECTORS**

**TO
PIXELS**

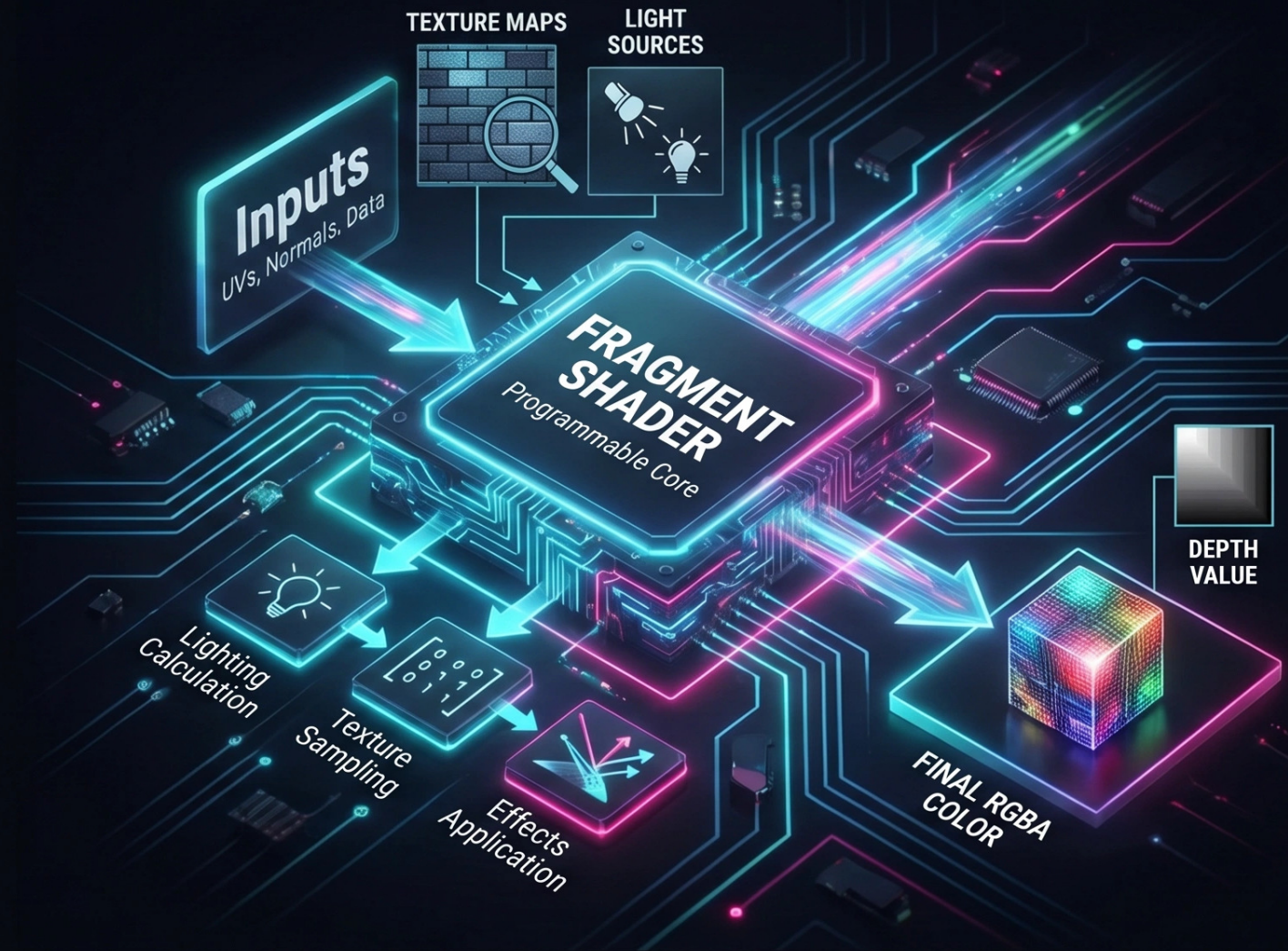


Stage 6: **Fragment Shader** — Computing Every Pixel's Color

The Final Step **Before the Screen**

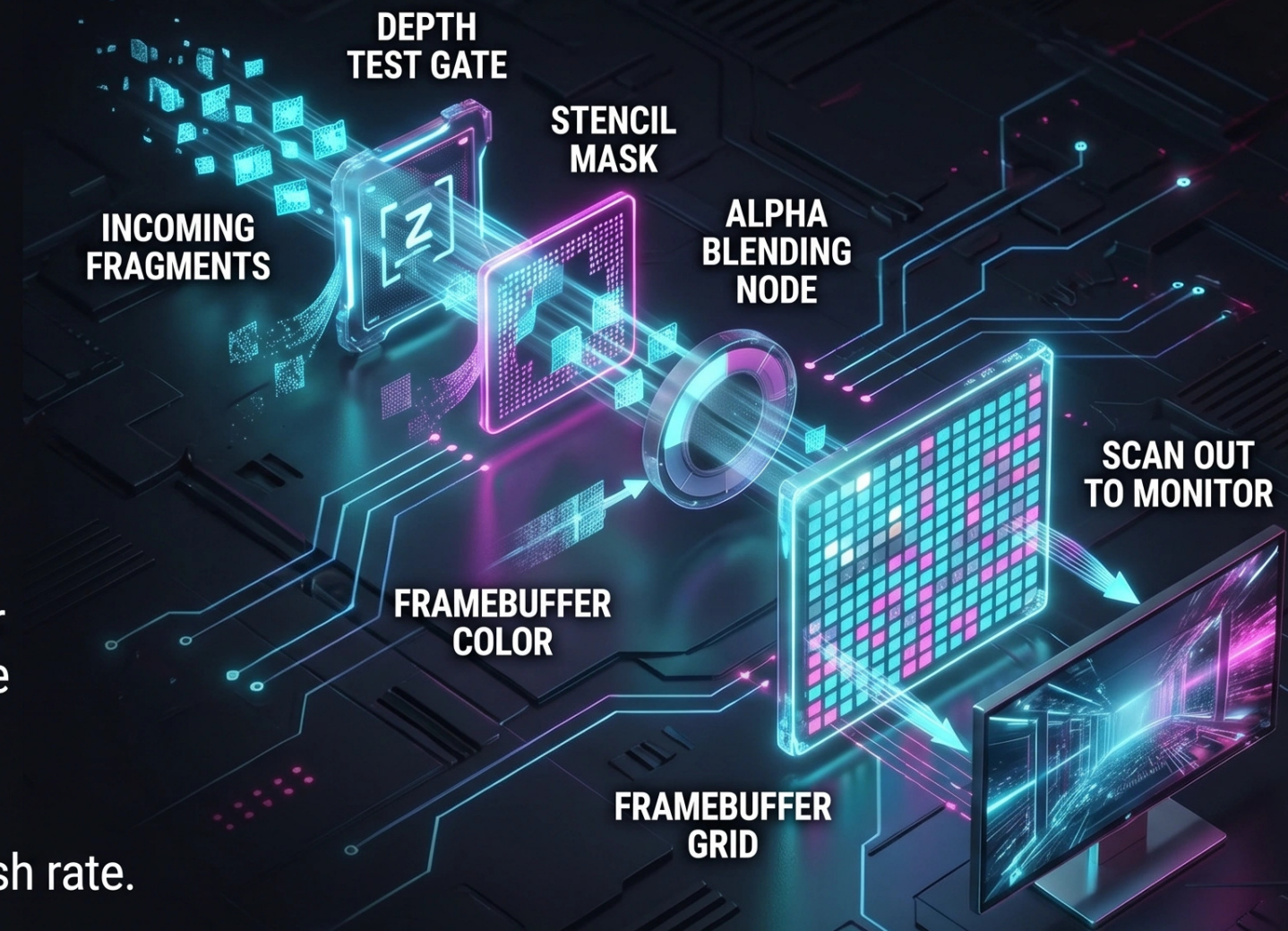
Key Points:

-  1. The most computationally intensive programmable stage — executes once per fragment (potential pixel).
-  2. Samples textures using interpolated UV coordinates for surface detail.
-  3. Computes lighting using models like Phong, Blinn-Phong, or PBR.
-  4. Applies effects: normal mapping, shadow mapping, ambient occlusion, reflections, transparency.
-  5. Output: final RGBA color and optionally a modified depth value.



STAGE 7: OUTPUT MERGER – THE FINAL GATEKEEPER.

- 1. Performs **depth testing (Z-buffer)**: compares fragment depth against stored depth – closer fragments win.
- 2. Executes **stencil testing**: masks regions of the screen for effects like mirrors, portals, and outlines.
- 3. Applies **alpha blending**: combines semi-transparent fragment colors with the existing framebuffer color.
- 4. Writes the surviving fragment's color to the framebuffer – the final 2D image stored in VRAM.
- 5. The display controller scans out the framebuffer to the monitor at the refresh rate.



HOW A SINGLE PIXEL IS BORN: END-TO-END WALKTHROUGH



Step 1: Application submits a draw call.



Step 2: **Input Assembler** reads vertex/index data and assembles triangles.



Step 3: **Vertex Shader** transforms each vertex to screen space.



Step 4: **Rasterizer** determines which pixels the triangle covers.

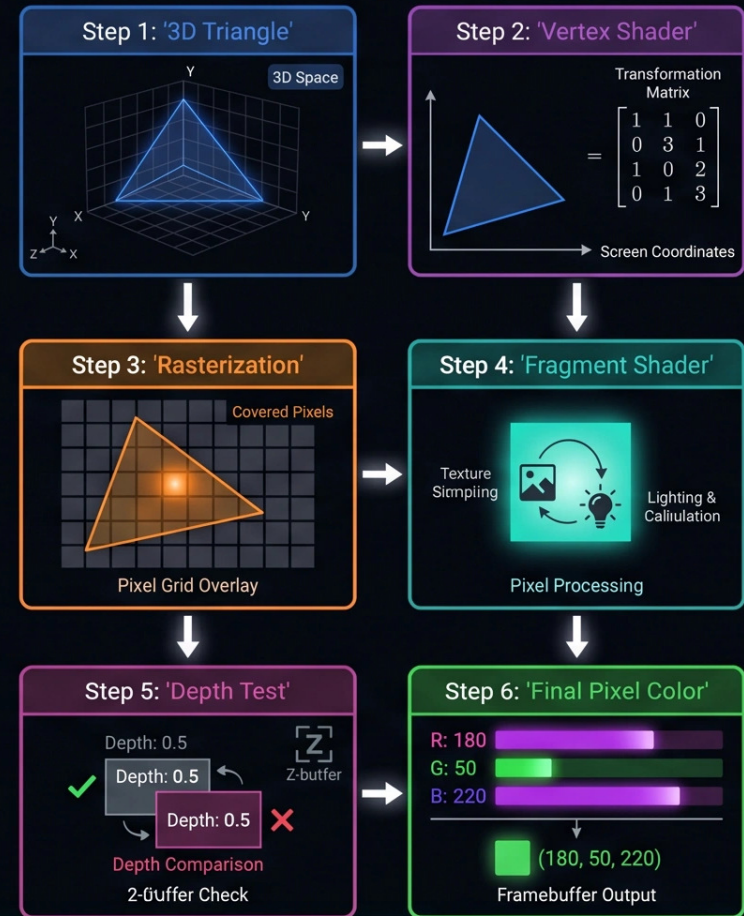


Step 5: **Fragment Shader** computes the color of each fragment.



Step 6: **Output Merger** performs depth test and writes the winning color to the framebuffer.

THE PIXEL GENERATION PIPELINE



GPU COMPUTE: BEYOND GRAPHICS

Unlocking General Purpose Processing Power.

1. GPGPU PARALLEL PROCESSORS

Modern GPUs are general-purpose parallel processors — compute shaders bypass the graphics pipeline entirely.

2. 3D GRID DISPATCH

Compute shaders dispatch thread groups (workgroups) organized in a 3D grid structure.

3. FAST LOCAL MEMORY

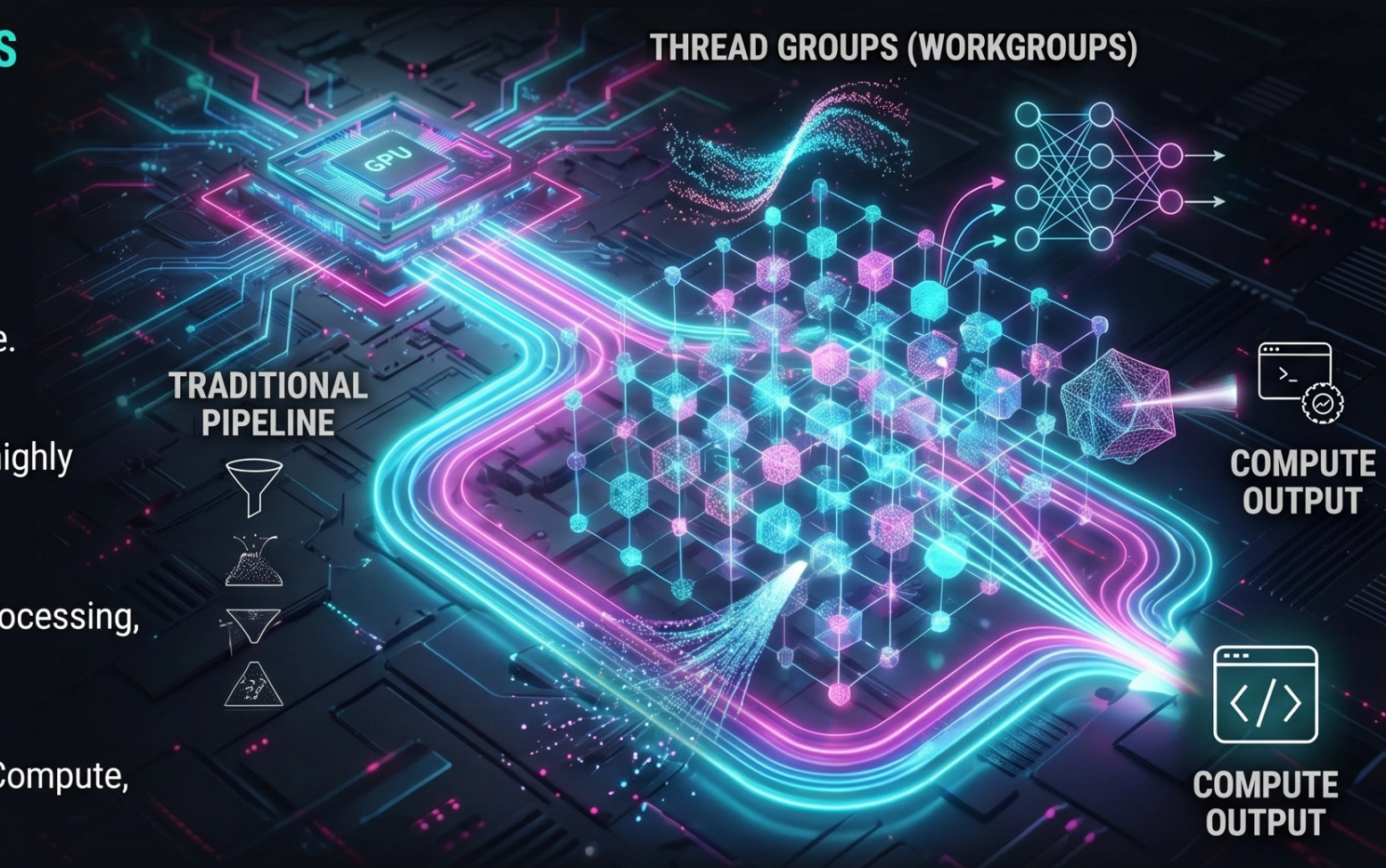
Each thread group shares local memory for highly efficient inter-thread communication.

4. WIDE APPLICATIONS

Physics simulation, particle systems, post-processing, AI inference, ray tracing, video encoding.

5. KEY APIS

CUDA, ROCm/HIP, Vulkan Compute, DirectX Compute, Metal Compute, OpenCL.



MODERN GPU FEATURES: RAY TRACING AND AI ACCELERATION

RT Cores, Tensor Cores, Mesh Shaders, and Variable Rate Shading: The Convergence of Rasterization, Ray Tracing, and AI.

- 1. **RT Cores**: Dedicated hardware for BVH traversal and ray-triangle intersection — accelerating ray tracing by 10x.
- 2. **Tensor Cores**: Matrix multiply-accumulate units for AI workloads — power DLSS/FSR upscaling and denoising.
- 3. **Mesh Shaders**: Replace traditional vertex/geometry pipeline with a flexible, compute-like model.
- 4. **Variable Rate Shading**: Reduces fragment shader invocations in less-important regions.
- 5. **Convergence of rasterization, ray tracing, and AI** in modern rendering.



SUMMARY: THE GPU AS A MASSIVELY PARALLEL ENGINE

- GPUs achieve throughput through **massive parallelism** — thousands of cores executing simple operations simultaneously.
- The **graphics pipeline** is a carefully orchestrated sequence of programmable and fixed-function stages.
- Every pixel on screen is the result of **vertex** transformation, rasterization, fragment shading, and depth testing.
- **Compute shaders** extend GPU capability far beyond graphics into general-purpose parallel computing.
- Understanding GPU architecture is fundamental for graphics programming, game development, and high-performance computing.

